

PROJECT WORK

SHOPPING CART

CODE:

```
import java.util.*;

// Abstract Superclass
abstract class User {
    protected String name; // common field

    public User(String name) {
        this.name = name;
    }

    // abstract method to be overridden (runtime polymorphism)
    public abstract void showMenu();
}

// Product Class
class Product {
    private final int id;
    private final String name;
    private final double price;
    private int stock;

    public Product(int id, String name, double price, int stock) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public double getPrice() { return price; }
    public int getStock() { return stock; }
```

```
public void reduceStock(int qty) { if (qty <= stock) stock -= qty; }
public void increaseStock(int qty) { stock += qty; }
```

```
@Override
public String toString() {
    return id + ". " + name + " - ₹" + price + " (Stock: " + stock + ")";
}
}
```

// OrderItem Class

```
class OrderItem {
    private final Product product;
    private int quantity;

    public OrderItem(Product product, int quantity) {
        this.product = product;
        this.quantity = quantity;
    }

    public Product getProduct() { return product; }
    public int getQuantity() { return quantity; }
    public void increaseQuantity(int qty) { this.quantity += qty; }
    public void decreaseQuantity(int qty) { this.quantity -= qty; }
    public double getSubtotal() { return product.getPrice() * quantity; }
}
```

// Cart Class

```
class Cart {
    private final List<OrderItem> items = new ArrayList<>();

    public void addToCart(Product product, int qty) {
        for (OrderItem item : items) {
            if (item.getProduct().getId() == product.getId()) {
                item.increaseQuantity(qty);
                return;
            }
        }
        items.add(new OrderItem(product, qty));
    }
}
```

```

public void removeFromCart(Product product, int qty) {
    Iterator<OrderItem> it = items.iterator();
    while (it.hasNext()) {
        OrderItem item = it.next();
        if (item.getProduct().getId() == product.getId()) {
            if (qty >= item.getQuantity()) it.remove();
            else item.decreaseQuantity(qty);
            return;
        }
    }
}

public boolean isEmpty() { return items.isEmpty(); }

public void viewCart() {
    if (items.isEmpty()) {
        System.out.println("Cart is empty.");
        return;
    }
    System.out.println("\n===== Cart =====");
    System.out.printf("%-15s %-10s %-10s %-10s\n", "Product", "Qty", "Price",
"Subtotal");
    double total = 0;
    for (OrderItem item : items) {
        System.out.printf("%-15s %-10d %-10.2f %-10.2f\n",
            item.getProduct().getName(),
            item.getQuantity(),
            item.getProduct().getPrice(),
            item.getSubtotal());
        total += item.getSubtotal();
    }
    System.out.println("-----");
    System.out.println("Cart Total = ₹" + total);
}

public List<OrderItem> getItems() {
    return new ArrayList<>(items);
}
}

// ===== Order Class =====

```

```

class Order {
    private final String buyerName;
    private final List<OrderItem> items;
    private final String location;

    public Order(String buyerName, List<OrderItem> items, String location) {
        this.buyerName = buyerName;
        this.items = items;
        this.location = location;
    }

    public void displayOrder() {
        System.out.println("\n==== Order Summary =====");
        System.out.println("Buyer: " + buyerName);
        System.out.println("Delivery Location: " + location);
        System.out.println("-----");
        System.out.printf("%-15s %-10s %-10s %-10s\n", "Product", "Qty", "Price",
"Subtotal");
        System.out.println("-----");
        double total = 0;
        for (OrderItem item : items) {
            System.out.printf("%-15s %-10d %-10.2f %-10.2f\n",
                item.getProduct().getName(),
                item.getQuantity(),
                item.getProduct().getPrice(),
                item.getSubtotal());
            total += item.getSubtotal();
        }
        System.out.println("-----");
        System.out.printf("Total Bill: ₹%.2f\n", total);
        System.out.println("=====");
    }
}

// ===== Seller Class (inherits User) =====
class Seller extends User {
    public Seller(String name) {
        super(name);
    }
}

// method overloading – addProduct (compile-time polymorphism)

```

```
public void addProduct(List<Product> store, int id, String name, double price) {
    addProduct(store, id, name, price, 5); // default stock 5
}
```

```
public void addProduct(List<Product> store, int id, String name, double price, int
stock) {
    store.add(new Product(id, name, price, stock));
    System.out.println(this.name + " added product: " + name);
}
```

```
@Override
public void showMenu() { // runtime polymorphism
    System.out.println("Seller Menu: Add and Manage Products");
}
}
```

// ===== Buyer Class (inherits User) =====

```
class Buyer extends User {
    private final Cart cart = new Cart();
```

```
public Buyer(String name) {
    super(name);
}
```

```
// method overloading – addToCart with default qty = 1
public void addToCart(List<Product> store, Scanner sc) {
    System.out.print("Enter product ID: ");
    int id = sc.nextInt();
    System.out.print("Enter quantity: ");
    int qty = sc.nextInt();
    addToCart(store, id, qty);
}
```

```
public void addToCart(List<Product> store, int id) {
    addToCart(store, id, 1);
}
```

```
public void addToCart(List<Product> store, int id, int qty) {
    Product found = null;
    for (Product p : store) {
        if (p.getId() == id) { found = p; break; }
```

```

    }
    if (found != null) {
        if (found.getStock() >= qty) {
            found.reduceStock(qty);
            cart.addToCart(found, qty);
            System.out.println("Added " + qty + " x " + found.getName() + " to cart.");
        } else {
            System.out.println("Only " + found.getStock() + " available in stock.");
        }
    }
    } else System.out.println("Invalid product ID.");
}

```

```

public void removeFromCart(List<Product> store, Scanner sc) {
    System.out.print("Enter product ID to remove: ");
    int id = sc.nextInt();
    System.out.print("Enter quantity to remove: ");
    int qty = sc.nextInt();

    Product found = null;
    for (Product p : store) {
        if (p.getId() == id) { found = p; break; }
    }

    if (found != null) {
        cart.removeFromCart(found, qty);
        found.increaseStock(qty);
        System.out.println("Removed " + qty + " from cart.");
    }
}

```

```

public void viewCart() { cart.viewCart(); }

```

```

public Order checkout(Scanner sc) {
    if (cart.isEmpty()) {
        System.out.println("Cart is empty. Cannot checkout.");
        return null;
    }
    sc.nextLine(); // clear buffer
    System.out.print("Enter delivery location: ");
    String location = sc.nextLine();
    return new Order(this.name, cart.getItems(), location);
}

```

```

}

@Override
public void showMenu() { // runtime polymorphism
    System.out.println("Buyer Menu: Browse and Purchase Products");
}
}

// ===== Main Class =====
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Product> store = new ArrayList<>();

        // Demonstrate polymorphism
        Seller seller = new Seller("Seller1");
        Buyer buyer = new Buyer("Aakash");

        seller.showMenu();
        buyer.showMenu();

        seller.addProduct(store, 1, "Laptop", 50000, 5);
        seller.addProduct(store, 2, "Mobile", 20000, 10);
        seller.addProduct(store, 3, "Headphones", 1500); // overloaded method (default
stock)

        while (true) {
            System.out.println("\n===== Shopping Menu =====");
            System.out.println("1. View Products");
            System.out.println("2. Add to Cart");
            System.out.println("3. Remove from Cart");
            System.out.println("4. View Cart");
            System.out.println("5. Checkout");
            System.out.println("6. Exit");
            System.out.print("Enter choice: ");
            int ch = sc.nextInt();

            switch (ch) {
                case 1:
                    System.out.println("\nAvailable Products:");
                    for (Product p : store) System.out.println(p);

```

```
        break;
    case 2:
        buyer.addToCart(store, sc);
        break;
    case 3:
        buyer.removeFromCart(store, sc);
        break;
    case 4:
        buyer.viewCart();
        break;
    case 5:
        Order order = buyer.checkout(sc);
        if (order != null) order.displayOrder();
        break;
    case 6:
        System.out.println("Thank you! Visit again!");
        sc.close();
        return;
    default:
        System.out.println("Invalid choice!");
    }
}
}
```